

AI Resume Builder

A rigorous technical study guide for interviews, demonstrations, and deeper software-engineering understanding

System in one sentence: A React application sends candidate information to a Spring Boot API that generates resumes, ATS feedback, cover letters, stored resume records, and PDF downloads.

Layer	Technology	Reason it exists
Presentation	React 19, Bootstrap, Axios	Captures inputs, manages UI state, calls APIs, and downloads files.
Application	Spring Boot 3, Java 21	Exposes REST endpoints and organizes use cases.
Persistence	Spring Data JPA, MySQL	Maps resume data to relational records and supports CRUD.
AI	Spring AI, Groq OpenAI-compatible API, Llama model	Creates resume text, feedback, and cover letters.
Export	OpenPDF	Creates PDF bytes delivered to the browser.

Prepared from the project source. No credentials are reproduced in this guide.

How to use this guide

This guide explains what the current project does, why its parts exist, and how to describe it at an advanced level. Study it once in order. Then revisit the architecture, risks, and interview sections until you can trace any user action from the browser to the backend and back.

Learning outcomes

- Explain the complete request flow from a React form submission to an AI response or a downloaded PDF.
- Distinguish controller, service, repository, entity, and DTO responsibilities in Spring Boot.
- Describe prompts, model integration, CORS, HTTP file delivery, and database persistence.
- Identify limitations in the code without overstating features that do not yet exist.
- Propose security, reliability, and product improvements appropriate for production.

Guide map

Part	Focus	Core question
1	System model	What does the application do and how do the pieces communicate?
2	React frontend	How are inputs, asynchronous calls, and downloads handled?
3	Spring Boot	How are endpoints and application layers structured?
4	Data and AI	How are data models, persistence, prompts, and LLM output handled?
5	Documents and integration	How do PDF export, CORS, validation, and errors work?
6	Production thinking	How would you secure, test, deploy, and improve the project?
7	Interview preparation	How should you present the project with technical confidence?

Important mindset: A strong answer explains the existing implementation and then identifies a precise improvement. Do not claim that a planned feature is already present.

1. System model and request lifecycle

The application is a client-server system. The browser hosts a React single-page interface. React sends HTTP requests to a Java Spring Boot API. The backend handles deterministic operations such as persistence and PDF generation, and probabilistic operations such as drafting a resume with a language model. MySQL is used to persist structured resume records.

End-to-end flow

Step	Component	Action	Output
1	Candidate	Completes name, contact, education, skills, experience, and projects.	Structured input.
2	React App	Stores input in state and sends Axios requests.	HTTP request to port 8080.
3	Controller	Maps endpoint and request body to a use case.	Service call.
4	Service	Builds prompt, uses repository, or creates PDF.	Text, record, or byte array.
5	Provider or database	LLM writes content; MySQL stores data; OpenPDF renders document.	Response payload.
6	React App	Updates screen state or downloads a Blob.	Visible text or resume.pdf.

Main user journeys

- Generate resume: form data -> POST /ai/generate -> prompt -> model -> generated text in the resume panel.
- Analyze resume: generated text -> POST /ai/analyze as text/plain -> analysis prompt -> model -> ATS report panel.
- Generate cover letter: form data -> POST /ai/cover-letter -> focused prompt -> model -> cover-letter panel.
- Download PDF: generated text -> POST /pdf/download as text/plain -> OpenPDF -> binary response -> browser download.

Key distinction: AI endpoints produce non-deterministic text. The PDF endpoint does not call the model; it deterministically turns supplied text into a document.

2. Frontend engineering: React state and HTTP

The frontend uses a React functional component with hooks. Form data lives in one state object, and each AI output has its own state variable. This is a practical shape for a compact form because the complete data object can be sent in one request.

State model

```
const [form, setForm] = useState({
  name: "", email: "", phone: "", education: "",
  skills: "", experience: "", projects: ""
});
const [resume, setResume] = useState("");
const [analysis, setAnalysis] = useState("");
const [coverLetter, setCoverLetter] = useState("");
const [loading, setLoading] = useState(false);
```

The form state represents what the candidate enters. Resume, analysis, and coverLetter represent results from separate API use cases. The loading flag prevents repeated actions while a call is in progress. This matters because language-model requests are slower and less predictable than a local calculation.

Generic change handler

```
const handleChange = (e) => {
  setForm({
    ...form,
    [e.target.name]: e.target.value,
  });
};
```

The spread operator makes a new object and the computed property name updates only the field identified by the input name. This avoids mutating existing state and enables React to rerender reliably.

Controlled input observation

The email field is explicitly controlled with `value={form.email}`, whereas several other inputs only provide `onChange`. Both can capture values, but a consistent controlled approach is easier to reset, validate, prefill, and test. A refinement should bind every input's value to its corresponding form property.

2.1 Async calls and browser file downloads

Generating a resume

```
const response = await axios.post(
  "http://localhost:8080/ai/generate",
  form
);
setResume(response.data);
```

Axios serializes the form object as JSON. Spring Boot deserializes it into `ResumeRequest`. The model returns plain text, so the frontend receives it in `response.data` and stores it in `resume` state. The `try/catch/finally` pattern should always reset loading, regardless of success or failure.

Why plain text endpoints set Content-Type

The analysis and PDF endpoints receive the generated resume itself, not a JSON object. The browser therefore sends `Content-Type: text/plain`. This communicates the body format to Spring's request handling layer and prevents accidental JSON parsing assumptions.

Binary PDF workflow

```
const response = await axios.post(
  "http://localhost:8080/pdf/download",
  resume,
  { headers: { "Content-Type": "text/plain" }, responseType: "blob" }
);
const url = window.URL.createObjectURL(response.data);
```

A PDF is binary data. `responseType: blob` stops Axios from parsing it as text or JSON. The application creates an in-memory object URL, gives it to a temporary anchor element, triggers a click, removes the element, and revokes the URL to release browser memory.

Interview phrasing: The application handles AI output as normal text state, but handles a PDF as binary data. That difference is why the client uses a Blob response type for file downloads.

3. Backend engineering: layered Spring Boot design

Spring Boot is organized into layers because HTTP concerns, business rules, database access, and data representation evolve differently. Controllers define the web contract. Services implement application use cases. Repositories abstract persistence. Entities describe stored domain data. DTOs describe API payloads.

Layer	Project classes	Primary responsibility	Avoid putting here
Controller	AIController, ResumeController, PdfController	Map paths and methods; parse input; return HTTP responses.	Complex business or database logic.
Service	AIService, ResumeService, PdfService	Run use cases and coordinate dependencies.	Frontend-specific rendering state.
Repository	ResumeRepository	Read and write Resume entities via JPA.	Prompts or document formatting.

Layer	Project classes	Primary responsibility	Avoid putting here
Entity	Resume	Map persisted resume fields to a database table.	A general-purpose API contract.
DTO	ResumeRequest	Carry AI-generation input including projects.	Persistence identity and database rules.

REST mappings

AIController is rooted at /ai, making its generation method POST /ai/generate. Its analysis and cover-letter methods become POST /ai/analyze and POST /ai/cover-letter. PdfController exposes POST /pdf/download. ResumeController exposes POST /resume, GET /resume, PUT /resume/{id}, and DELETE /resume/{id}.

Dependency injection

Spring creates application objects, known as beans, and injects required dependencies. The project uses @Autowired field injection in several classes. AIService uses constructor injection, which is preferable because its dependencies are explicit, can be final, and are easier to provide in tests.

Preferred refactor: Use constructor injection consistently. It makes a class's requirements visible and avoids mutable injection fields.

4. Data model, DTOs, and MySQL persistence

The Resume entity

Resume is marked with @Entity and mapped to the resume table. Its id uses GenerationType.IDENTITY, so the database creates unique primary-key values. The current persisted attributes are name, email, phone, skills, education, and experience.

```
@Entity
@Table(name = "resume")
public class Resume {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String phone;
    private String skills;
    private String education;
    private String experience;
}
```

Why ResumeRequest is valuable

The generation request contains a projects field, but the database entity does not. This is a concrete reason to separate API DTOs from entities: the contract required to generate a document is not necessarily identical to the data currently persisted in a table.

CRUD flow

ResumeService calls repository.save to insert or update records and repository.findAll to list them. updateResume first loads an existing object with findById, copies selected properties, and saves it. deleteResume calls deleteById. JPA translates these operations into SQL.

Current data-model gaps

- Projects are used in AI prompts but are not stored on Resume or copied during update.
- There is no GET /resume/{id} endpoint for fetching one record.

- There are no validation constraints for required fields, email syntax, or input length.
- Missing ids are not converted into clear 404 responses.
- A future product may model repeated employment, education, and project records as ordered child entities.

Design principle: Do not normalize a database just for formality. Use structured child tables when they enable better queries, consistency, revisions, or user-facing features.

5. AI integration and prompt engineering

AIService wraps Spring AI's ChatClient. It receives a prompt, sends it to the configured chat model, performs a synchronous call, and returns the content. The configuration points Spring AI at a Groq OpenAI-compatible endpoint using a Llama-family model.

```
public String generateResume(String prompt) {  
    return chatClient.prompt(prompt)  
        .call()  
        .content();  
}
```

Prompt construction

The controller combines a task instruction with labeled candidate details. Resume generation asks for an ATS-friendly resume. Resume analysis asks for an ATS score, missing skills, grammar suggestions, improvements, and overall feedback. Cover-letter generation currently targets a Java Spring Boot Developer position.

Prompt element	Current purpose	Stronger version
Task instruction	Tells the model to generate or analyze.	Specify seniority, tone, output sections, and constraints.
Candidate fields	Gives facts to base output on.	Validate fields and show omitted values clearly.
ATS direction	Guides screening-focused language.	Include a target job description and requested keywords.
Analysis list	Encourages separate feedback topics.	Require JSON for reliable interface rendering.
Fixed role	Makes the cover-letter demo focused.	Accept title, company, and job description dynamically.

Hallucination and factual accuracy

A language model predicts plausible text, not verified facts. It may invent metrics, credentials, employers, or skills. The system should treat generation as a draft based on user-supplied facts. Users must review every claim before submitting the resume.

Strong engineering statement: The AI should improve phrasing, organization, and relevance. It should not invent a candidate's employment history, qualifications, achievements, or technical skills.

5.1 Production-grade AI design

Structured output

Free-form text is easy to demonstrate but difficult for a user interface to interpret reliably. For ATS analysis, the better contract is a JSON object with a numeric score and arrays for missing skills, grammar suggestions, and improvements. The backend can validate the response before returning it.

```
{  
  "score": 78,  
  "missingSkills": ["Docker", "AWS"],  
  "grammarSuggestions": ["Replace passive wording in experience item 2"],  
}
```

```
"improvements": ["Add measurable outcome metrics"],
"overallFeedback": "Strong entry-level backend profile."
}
```

Reliability controls

- Set connection and read timeouts so an unavailable model cannot block the application indefinitely.
- Rate-limit requests to manage AI cost and prevent abuse.
- Map upstream provider failures to safe application errors such as 502 or 503.
- Log model version, prompt version, latency, and error category without logging sensitive resume content by default.
- Use asynchronous jobs for expensive generation flows as traffic increases.

Privacy and prompt injection

A resume contains personal data, and future job descriptions or uploads may contain adversarial instructions. Treat all user-supplied content as data inside a clear prompt boundary. Minimize retention, disclose external AI processing, and never allow user text to override the system's safety and formatting rules.

6. PDF generation and HTTP delivery

PdfService uses OpenPDF to create a Document, attach a writer to a ByteArrayOutputStream, add paragraphs, close the document, and return a byte array. PdfController sets the HTTP headers and body. The browser receives the bytes as a Blob and downloads resume.pdf.

```
return ResponseEntity.ok()
    .header(HttpHeaders.CONTENT_DISPOSITION,
        "attachment; filename=resume.pdf")
    .contentType(MediaType.APPLICATION_PDF)
    .body(pdf);
```

Element	Meaning	Browser impact
Content-Type: application/pdf	Identifies the response MIME type.	The response is treated as a PDF binary file.
Content-Disposition: attachment	Requests a download and suggests a filename.	The browser downloads resume.pdf.
ResponseEntity<byte[]>	Carries status, headers, and binary body.	Spring writes a complete PDF response.
Axios Blob response	Preserves binary data on the client.	JavaScript can create a temporary download URL.

Current and improved rendering

The current PDF has a title and raw resume text. This demonstrates a correct transport pipeline, but it is not yet a sophisticated resume renderer. A better design returns structured resume data from the AI layer and renders headings, bullets, contact details, experience entries, and page-aware layout with a selected template.

- Keep content text-based and use conventional headings so applicant-tracking systems can read the PDF.
- Use a registered Unicode font if the application accepts names or content outside ASCII.
- Control margins, typography, bullet indenting, and page breaks for professional presentation.
- Stream large document output rather than creating one large byte array in memory.

7. CORS, validation, and error handling

CORS

React normally runs at `http://localhost:3000` in development while Spring Boot runs at `http://localhost:8080`. The different ports make them different origins. `@CrossOrigin` on `AIController` and `PdfController` permits the React origin to call the API. CORS is a browser rule; it is not authentication.

In production, replace the local development origin with the exact deployed frontend domain. Do not use broad wildcard origins for sensitive authenticated applications.

Validation

Current request bodies are accepted without validation. Add `jakarta.validation` constraints to `ResumeRequest` and use `@Valid` on `@RequestBody` arguments. This rejects blank names, malformed emails, and oversized inputs before the application calls a paid AI service or writes the database.

```
public class ResumeRequest {
    @NotBlank @Size(max = 100) private String name;
    @NotBlank @Email private String email;
    @Size(max = 4000) private String experience;
}

public String generateResume(@Valid @RequestBody ResumeRequest request) { ... }
```

Exception handling

`updateResume` currently calls `orElseThrow` without a domain-specific exception. A missing id can become an unclear server error. A `@RestControllerAdvice` should translate expected failures into useful responses.

Failure	Recommended status	Response idea
Resume id absent	404 Not Found	Resume with id 42 does not exist.
Invalid request	400 Bad Request	email must be a valid email address.
Model provider timeout	503 Service Unavailable	Generation service is temporarily unavailable.
PDF rendering failure	500 Internal Server Error	Could not create PDF; try again.

8. Security and privacy review

The application processes names, contact information, education, and work history. It also uses a database and an external AI provider. A learning prototype can demonstrate integration, but a deployable application must protect secrets, establish record ownership, limit access, and disclose data processing.

Immediate action: The project configuration contains secrets directly in source configuration. Treat them as exposed: rotate the database password and AI key, remove them from source control, and replace them with environment variables or a managed secret store.

Safe configuration pattern

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.ai.openai.api-key=${GROQ_API_KEY}
```

Security checklist

- Use `.gitignore` and an example configuration file containing placeholders only.
- Use profiles or deployment variables for development, test, and production configuration.

- Add authentication before storing or retrieving real user records.
- Authorize by ownership: a user must not access a resume by guessing its id.
- Use HTTPS in production and apply safe headers.
- Set request-size limits, rate limits, and quotas for AI endpoints.
- Minimize transfer and retention of personal data; document how the AI provider processes it.
- Keep dependencies current and scan for vulnerabilities.

Deleting a secret from the current file is not enough if the project was committed or shared. Source history, archives, build logs, and forks may retain it. Rotation invalidates the old secret and is the correct remediation.

9. Testing, operations, and deployment

Test strategy

Test level	Target	Example assertion
Unit	ResumeService	Updating a record copies all intended fields and missing ids become domain errors.
Web slice	AIController	POST /ai/generate maps JSON to ResumeRequest and calls the service.
Repository	ResumeRepository	Entity mappings and CRUD work with a test database.
Integration	Whole API	Endpoints return correct status, headers, bodies, and errors.
Frontend	React components	Mock Axios, fill form, click button, and assert loading, result, and errors.

Operational concerns

- Expose health checks using Spring Boot Actuator for deployment monitoring.
- Log endpoint, latency, status, and upstream failure category while redacting PII and secrets.
- Measure generation latency, error rate, model cost, request volume, and PDF failures.
- Use Flyway or Liquibase for schema migrations rather than relying on ddl-auto=update in production.
- Deploy React as static assets, Spring Boot as an API service, and MySQL as a managed database with environment-specific configuration.

Production principle: A working demo proves functionality. A production service additionally proves security, resilience, ownership, observability, testing, and repeatable deployment.

10. Technical roadmap and code-review findings

The existing project is a readable and valuable prototype. The roadmap below addresses the most important gaps first, preserving the simple learning-oriented structure while moving toward a real product.

Priori ty	Finding	Impact	Suggested change
P0	Secrets in configuration	Provider and database compromise risk.	Rotate secrets and use environment variables.
P1	No validation or error advice	Bad inputs and absent ids yield unclear failures.	Use @Valid and @RestControllerAdvice.

Priority	Finding	Impact	Suggested change
P1	No authentication or ownership	Records could be exposed or modified by other callers.	Add users, authentication, and per-user authorization.
P1	Projects not persisted	AI input is not fully saved or updated.	Add projects field or normalized project records.
P2	Free-form AI analysis	Interface cannot reliably render individual findings.	Use validated structured JSON output.
P2	Simple PDF layout	Document is functional but not portfolio-ready.	Render structured sections from a template.
P2	Field injection	Dependencies are less explicit and testable.	Adopt constructor injection.
P3	Hardcoded frontend URLs	Deployment needs code edits.	Use environment-specific frontend configuration.

Feature sequence

- Phase 1: credential rotation, validation, exception handling, constructor injection, and action-specific loading states.
- Phase 2: job-description input, keyword matching, editable output, and professional ATS-safe PDF templates.
- Phase 3: user accounts, saved dashboards, revision history, secure ownership, and export history.
- Phase 4: structured AI output, configurable roles and tones, provider fallback, and cost observability.

11. Interview preparation

A 60-second project explanation

Suggested answer: I built an AI Resume Builder with a React frontend and Spring Boot backend. A candidate enters profile, education, skills, experience, and projects. React sends that information to REST endpoints. The backend uses Spring AI with an OpenAI-compatible language-model provider to generate an ATS-friendly resume, analyze it, and create a cover letter. I also added JPA and MySQL CRUD support plus PDF export with OpenPDF. The main engineering topics are asynchronous UI state, REST design, prompt engineering, CORS, binary file downloads, database persistence, and secure handling of personal data.

Core interview questions

Question	Strong answer direction
Why React and Spring Boot?	React is effective for interactive form workflows. Spring Boot provides REST, DI, JPA, validation, test support, and AI integration.
How does data flow?	Form state -> Axios -> controller -> service -> model, database, or PDF renderer -> response -> React state or Blob download.
Why use a DTO?	ResumeRequest represents API needs, including projects, independently of the persisted Resume entity.
Why use Blob for PDFs?	A PDF is binary. Blob preserves raw bytes and enables a browser object URL download.
What does CORS do?	It permits the development React origin to call a backend on a different port. It does not authenticate callers.
How would you improve AI quality?	Use job descriptions, schema-constrained output, factual-claim checks, prompt versioning, and an editable review step.
Most urgent security fix?	Rotate exposed configuration secrets, then use environment variables and add authenticated record ownership.

11.1 Advanced discussion answers

Why not call a repository directly from a controller?

A controller should be a transport adapter: it maps HTTP to a use case and results back to HTTP. Business rules commonly grow to include authorization, auditing, retries, provider fallback, and transactions. The service layer owns such rules and can be reused by HTTP, scheduled, or message-driven entry points.

How would you make an LLM product more predictable?

The exact wording remains probabilistic, but the system can be predictable around it: fixed prompt versions, controlled generation settings, JSON schema output, input validation, factual constraints, content review, editable user drafts, and telemetry that records model and prompt versions.

How would the database evolve?

Introduce a User entity and an ownership relationship to Resume. Retain resume metadata and version history, then decide whether experience, education, and projects belong in ordered child tables or validated JSON based on future queries. Use Flyway for migrations.

What happens when the model provider fails?

Map the provider exception to an application-level failure, apply timeouts, retry only transient failures, log internal details safely, and return a clear temporary-unavailable response. Optionally offer a retry path or carefully controlled fallback provider.

What makes an ATS-friendly PDF?

It should preserve selectable text, use clear conventional headings, avoid important content inside images or complex graphics, use standard fonts, maintain simple structure, and prioritize semantic readability over decorative layout.

12. Demonstration and revision checklist

Demonstration sequence

- Start MySQL and confirm the configured schema is available.

- Start Spring Boot with non-secret local configuration and confirm the AI provider is reachable.
- Start React, enter a realistic candidate profile, and generate a resume.
- Review generated statements for factual accuracy before analysis or export.
- Run ATS analysis and explain its five feedback categories.
- Generate a cover letter and note that the current prompt targets a Java Spring Boot Developer role.
- Download the PDF and confirm it opens as a valid file.
- If demonstrating CRUD endpoints, create, list, update, and delete a safe test record.

Before publishing or deploying

- Rotate exposed keys and passwords, and remove secrets from configuration and repository history.
- Commit an example configuration file with placeholders only.
- Add validation, standardized exception responses, and tests.
- Set production CORS origins and enable authentication before handling real personal data.
- Move database schema management to migrations and configure safe production settings.
- State clearly that every AI-generated claim requires user review.

Final takeaway: This project is a strong learning foundation because it combines modern frontend work, a layered Java backend, relational persistence, AI integration, and document generation. Moving from prototype to production mainly requires secure configuration, structured data, validation, ownership, resilience, and polished user experience.

End of study guide